Label-free resets for a continual duo: screening, tuning and a test-stream comparison

Dynamic Duos, ViT-B/16 + ResNet-50 V1, Tent, Vanilla DD

28 October 2026

1 Question and setting

A reset puts a member back to its source weights. We want a rule that flags a collapsing member soon and rarely resets a healthy one, using only the label-free signal the gate already computes. We compare 15 detector families on stored runs (the *screen*), then run the five best live with real resets (Section 7).

The setting is the one of the gate-stress experiments: Vanilla DD (each member adapts with Tent on its own entropy, the gate sits outside the loop), the nuclear-norm proxy with proxy batch $b_t = 256$, gate $\beta = 2$ (β changes neither the members nor the proxy values). In the screen and the first live runs the learning rates are *fixed* to values that collapse both members (stress pair: ViT-B/16 $1.4{\cdot}10^{-4}$, ResNet-50 $5{\cdot}10^{-4}$); the rates tuned for stability without resets (the 04b rates) are too low for a resetting scheme. Sections 8–10 then tune the rates and thresholds of every method on the held-out stream under fixed rules and compare the tuned methods once on the test stream.

2 The rules

Signals. Each proxy batch gives each member a proxy value $r_t^{(m)} \in (0, 1)$; let $s_t^{(m)} = \mathrm{logit}(r_t^{(m)})$ (clipped). A member's rule watches either its own score (*own*, $s_t^{(m)}$), its gap to the partner (*gap*, $s_t^{(m)} - s_t^{(m')}$, antisymmetric between the two members), or either of the two (*both*: gap OR own, same parameters).

Procedures. Five one-sided detectors, each standardized: the deviation is divided by a running standard deviation of the signal (exponential rate 0.01, taken before the new value enters), so thresholds are in noise units. For the first W_0 batches after (re)start a detector only learns its reference mean m_0. With σ the running standard deviation, a detector alarms when

- **level**: $(x_t - m_0)/\sigma < \tau$;
- **imm_vs_ema**: $\bar{x}_{t-1} - x_t > \delta\sigma$, with $\bar{x}$ an exponential average (rate α) started at m_0;
- **fast_vs_slow**: $\bar{x}_t^{\mathrm{slow}} - \bar{x}_t^{\mathrm{fast}} > \delta\sigma$, two exponential averages started at m_0;
- **slope**: the least-squares slope over the last W values, divided by σ, is below $-\gamma$ for k consecutive batches;
- **cusum**: $S_t = \max(0, S_{t-1} + (\mu - x_t)/\sigma - \nu) > h$, with μ the frozen warm-up mean (own) or 0 (gap).

Controller. A member is flagged when any detector of its rule alarms. A flagged member has its detectors restarted (and the partner's gap detector, since that is built from the flagged member's proxy), and cannot be flagged again for $K = W_0$ batches.

Grid. Table 1 lists the settings. Three signal choices × five procedures give 15 families; that makes 232 settings per signal choice (16 + 48 + 48 + 72 + 48) and **696** in total.

3 Data

Every run is Vanilla DD without any reset, stored together with a frozen-member run on the same images and the gate's per-batch proxy values. A rule is *replayed* on these proxy sequences exactly as the live controller would

Table 1: Grid of one family; every row is crossed with $W_0 \in \{20, 50\}$.

procedure	swept parameters	threshold	values
level	—	τ	$-1, -1.5, -2, -3, -4, -5, -6, -8$
imm_vs_ema	$\alpha \in \{0.01, 0.05, 0.1\}$	δ	$1, 1.5, 2, 3, 4, 5, 6, 8$
fast_vs_slow	$(\alpha_f, \alpha_s) \in \{(.05, .005), (.1, .01), (.2, .02)\}$	δ	$1, 1.5, 2, 3, 4, 5, 6, 8$
slope	$W \in \{16, 32, 64\}$, $k \in \{1, 4\}$	γ	$.005, .01, .02, .05, .1, .2$
cusum	$\nu \in \{0.25, 0.5, 1\}$	h	$2, 4, 8, 12, 16, 24, 32, 48$

run, but the trajectory is not changed (the reset model cannot be simulated offline), so only each member's *first* collapse is measured. The live runs rank the finalists.

Table 2: The three sets of runs. S: held-out stream (speckle, Gaussian blur, spatter, saturate; severity 5, 50k images each; 4 rounds, round r rotated by r), 4 rotations $\times$ 2 seeds. H: the same stream at full length, rotations 0 and 2, seed 0. T: the test stream, seeds 0–2. A member-batch is one proxy batch of one member (a stream contributes two); "healthy" = outside every degraded stretch (see below).

set	ViT rate	ResNet rate	streams	member-batches	healthy	events
S (calibration)	$1.4{\cdot}10^{-4}$	$5{\cdot}10^{-4}$	8	19,328	34%	23
H (calibration)	$1.26{\cdot}10^{-5}$	$2.2{\cdot}10^{-5}$	2	12,500	99%	3
T (test)	$1.4{\cdot}10^{-4}$	$5{\cdot}10^{-4}$	3	10,948	54%	9

S stops 20,000 images after both members are dead, so most of each S run is degraded: it supplies 23 of the 26 calibration events but only a third of the healthy time. H is the stable pair: it supplies the healthy time, and almost every alarm in it is false (its only events are 3 short dips of the ResNet-50). T is never used for a choice.

4 Labels and metrics

Labels are used only for evaluation; they never reach a rule.

- A member is *degraded* at batch t if it is clearly worse than its own frozen (non-adapting) copy on the same images. Three label parameters define "clearly":
  - M, the *smoothing window* (in batches): the accuracy difference "adapted minus frozen" is averaged over M batches. Default $M = 8$.
  - δ, the *margin* (in accuracy points; 1 pt = one percentage point): the member is degraded when that smoothed difference is below $-\delta$. Default $\delta = 1$ pt.
  - L, the *minimum stretch length* (in batches): a run of consecutive degraded batches counts only if it lasts at least L batches. Such a stretch is a *collapse event* and its first batch is the *onset*; shorter dips are ignored. Default $L = 16$.
- **Delay**: batches from the onset to the first alarm in the stretch (an alarm up to 8 batches before the onset counts as delay 0). A stretch with no alarm is a **miss** and is charged its whole length as delay.
- **False alarm**: an alarm outside every stretch (less the 8-batch slack). The **false-alarm rate** is false alarms per 1,000 member-batches, all batches counted (degraded ones too).

One batch is 256 images, so a delay of 100 batches is 25.6k images. A **false-alarm budget** of 1 per 1,000 allows one unneeded reset per member per 256k images (about 3 per member on an 800k-image stream). Without a budget the comparison is meaningless: a rule that fires every batch has zero delay. A budget fixes the cost of a reset on a healthy member and lets each family compete at its best setting.

5 Selection protocol

1. Replay all 696 settings on the **calibration pool**: S and H together (10 streams, 31,828 member-batches, 26 events: 8 ViT-B/16, 18 ResNet-50). Events, misses, delays, false alarms and batches are summed over all (stream, member) pairs.

2. For each family and each budget $b \in \{0.5, 1, 2\}$ keep the settings whose pooled false-alarm rate is at most b, and take the one with the lowest mean delay (ties: fewer misses, then fewer false alarms). One setting, one threshold, for both members.
3. **Freeze** that setting and score it once on the **test streams** T. Nothing is chosen on T and the budget is not enforced there: a test false-alarm rate above the budget measures how well the setting transfers.
4. Report the budget-1 table, the other budgets, per-member scores, and the sensitivity of the ranking to the labels ($M \in \{4, 8, 16\}$ batches, $\delta \in \{0.5, 1, 2\}$ pt, $L \in \{8, 16, 32\}$ batches; one parameter changed at a time).

6 Screening results

Budget 1 (Table 3). The three leaders on the calibration pool are `gap_fast_vs_slow`, `own_fast_vs_slow` and `own_cusum`. On the test streams `own_fast_vs_slow` is best: delay 23 batches (about 6k images), 2 of 9 events missed, 0.27 false alarms per 1,000. `gap_fast_vs_slow` has a similar delay but 1.19 false alarms on the test streams, above the budget. `both_cusum` and `gap_cusum` have no setting within any budget: their lowest false-alarm rate on the grid is 3.8 per 1,000, so their threshold grid is too low to conclude anything about the rule.

Table 3: Budget of 1 false alarm per 1,000 member-batches: the setting chosen on the calibration pool (S+H), scored there and, frozen, on the test streams T. Delay in 256-image batches (a miss is charged its whole length), miss = missed events / events, FA = false alarms per 1,000 member-batches. Sorted by calibration delay.

	calibration (26 events)			test (9 events)		
rule	delay	miss	FA	delay	miss	FA
`gap_fast_vs_slow`	110	0.35	0.91	32	0.33	1.19
`own_fast_vs_slow`	113	0.31	0.69	23	0.22	0.27
`own_cusum`	133	0.31	0.72	59	0.33	0.00
`own_level`	147	0.31	0.69	96	0.33	0.00
`both_fast_vs_slow`	153	0.38	0.72	450	0.67	0.00
`gap_slope`	156	0.38	0.31	553	1.00	0.09
`both_level`	170	0.42	0.82	120	0.56	0.18
`gap_level`	171	0.50	0.53	55	0.33	0.00
`own_imm_vs_ema`	203	0.54	0.97	448	0.56	0.64
`both_slope`	214	0.58	0.75	401	0.78	0.27
`own_slope`	215	0.62	0.75	529	0.89	0.27
`both_imm_vs_ema`	215	0.54	0.94	553	1.00	0.27
`gap_imm_vs_ema`	319	0.73	0.72	553	1.00	0.09
`both_cusum`	no setting within the budget					
`gap_cusum`	no setting within the budget					

Selected settings of the leaders (Table 4). The own and gap `fast_vs_slow` leaders use the slowest pair of averages of the grid and the lowest threshold; `both_fast_vs_slow` uses a faster pair and $\delta = 1.5$.

Table 4: Frozen settings of the five leaders at budget 1 and their calibration false-alarm rate.

rule	W_0	parameters	threshold	FA (calib.)
`gap_fast_vs_slow`	50	(α_f, α_s)=(0.05, 0.005)	$\delta = 1$	0.91
`own_fast_vs_slow`	20	(α_f, α_s)=(0.05, 0.005)	$\delta = 1$	0.69
`own_cusum`	50	ν=0.25	$h = 2$	0.72
`own_level`	50	—	$\tau = -1.5$	0.69
`both_fast_vs_slow`	20	(α_f, α_s)=(0.1, 0.01)	$\delta = 1.5$	0.72

Across budgets (Table 5). `fast_vs_slow` is the one procedure in the top five of the calibration pool at all three budgets (any signal choice). `imm_vs_ema` is the weakest family: it compares a sample with its own moving

average, so a slow ramp drags the average along and is not seen (the ResNet-50 decays gradually). `slope` is the best at budget 2 on the calibration pool but is erratic on the test streams (own_slope and gap_slope at budget 1 miss most or all test events). The ranking changes with the budget: `gap_fast_vs_slow` leads at 0.5 and 1, `own_slope` at 2 (it is 11th at budget 1). Settings that are best on the calibration pool can also miss on T: `both_fast_vs_slow` has delay 450 on T at budget 1 (its ResNet-50 events are missed) but 28 at budget 0.5.

Table 5: Mean delay (256-image batches, misses charged their whole length) of each family's chosen setting at the three budgets, on the calibration pool and, frozen, on the test streams. Best per calibration column in bold; – = no setting within the budget.

	calibration			test		
budget (FA per 1,000)	0.5	1	2	0.5	1	2
`own_level`	164	147	141	161	96	67
`gap_level`	232	171	119	85	55	31
`both_level`	296	170	143	220	120	24
`own_imm_vs_ema`	488	203	109	553	448	243
`gap_imm_vs_ema`	339	319	216	553	553	441
`both_imm_vs_ema`	488	215	215	553	553	553
`own_fast_vs_slow`	230	113	102	49	23	14
`gap_fast_vs_slow`	**147**	**110**	85	450	32	13
`both_fast_vs_slow`	154	153	74	28	450	19
`own_slope`	488	215	**54**	553	529	30
`gap_slope`	156	156	80	553	553	130
`both_slope`	488	214	116	553	401	457
`own_cusum`	149	133	133	121	59	59
`gap_cusum`	–	–	–	–	–	–
`both_cusum`	–	–	–	–	–	–

Per member (Table 6). The two `fast_vs_slow` leaders catch the ViT-B/16 within 11–32 batches (3k–8k images) on both calibration and test, and miss none of its long collapses; `own_cusum` and `own_level` take 21–85. The ResNet-50 decays gradually, so its onset (1 point below frozen) comes long before any detector can tell: calibration delays are around 150 batches, including the charge for missed dips. Missed events are almost all short: for `own_fast_vs_slow` the 8 calibration misses are ResNet-50 stretches of 16, 17, 26, 31, 43, 58, 97 and 265 batches, and the 2 test misses are ViT-B/16 dips of 35 and 38 batches; every stretch of 300 batches or more is detected. Whether a short dip that recovers on its own should trigger a reset is a modelling choice (see below).

Table 6: Per member at budget 1: mean delay in batches (missed / events). Calibration: 8 ViT-B/16 and 18 ResNet-50 events; test: 5 and 4.

	calibration		test	
rule	ViT-B/16	ResNet-50	ViT-B/16	ResNet-50
`gap_fast_vs_slow`	11 (0/8)	154 (9/18)	26 (2/5)	39 (1/4)
`own_fast_vs_slow`	32 (0/8)	149 (8/18)	28 (2/5)	18 (0/4)
`own_cusum`	21 (0/8)	183 (8/18)	62 (2/5)	56 (1/4)
`own_level`	22 (0/8)	203 (8/18)	85 (2/5)	111 (1/4)
`both_fast_vs_slow`	16 (0/8)	213 (10/18)	27 (2/5)	978 (4/4)

Label sensitivity (Table 7). Under M and the margin δ the delays move by at most 12%; `own_cusum` stays behind the two `fast_vs_slow` rules in every variant, and those two stay within 12 batches of each other. The minimum stretch length L moves the miss rate most, because it decides how many short dips are counted as events.

Table 7: The three leaders' frozen settings re-scored on the calibration pool under other degradation labels. Each row changes one label parameter of Section 4 from the base row: M = smoothing window (batches), δ = margin (accuracy points below the frozen member), L = minimum stretch length (batches). Cell = mean delay in 256-image batches / miss rate (missed events / events).

labels	`gap_fast_vs_slow`	`own_fast_vs_slow`	`own_cusum`
base (M=8, δ=1 pt, L=16)	110 / 0.35	113 / 0.31	133 / 0.31
$M = 4$	98 / 0.39	109 / 0.39	127 / 0.36
$M = 16$	115 / 0.35	118 / 0.31	139 / 0.31
$\delta = 0.5$ pt	99 / 0.45	101 / 0.42	118 / 0.42
$\delta = 2$ pt	100 / 0.37	99 / 0.33	119 / 0.33
$L = 8$	78 / 0.55	80 / 0.53	94 / 0.53
$L = 32$	125 / 0.23	129 / 0.18	153 / 0.18

7 Live runs with resets

The screen replays rules on stored runs, so it cannot show what a reset does to the rest of the stream. We therefore ran the five leaders of Table 4 live: their frozen settings drive a controller that, when it flags a member, restores that member's source weights and clears its optimizer before the next batch (the gate keeps its buffer and is not reset).

Setup. Vanilla DD with Tent, ViT-B/16 at $1.4{\cdot}10^{-4}$ and ResNet-50 at $5{\cdot}10^{-4}$ (the stress pair: without resets both collapse), $\beta = 2$, whole streams, no early stop. Streams: the held-out stream (4 shifts $\times$ 4 rounds; opening shift 0 and 2; one seed) and the test stream (15 corruptions, standard order; seeds 0–2, which change the data order only). Conditions:

- **No reset**, and the **frozen** duo (no adaptation) as the reference.
- The five **rules**, label-free.
- **Fixed schedule**: both members every N adaptation steps (batch size 64): $N = 250, 500, 1000, 2000$ on held-out, 500 and 1000 on test (1000 is RDumb's period; 500 was the best on held-out, by 0.1 point). Label-free and blind.
- **Episodic**: both members and the gate restart at every shift boundary, the episodic protocol run as one stream. It is told where the shifts are, which no rule is: a reference.
- **Oracle**: uses the labels and a frozen copy of the pair stepped in lockstep; a member is reset once its accuracy over the last 8 proxy batches has stayed more than 1 point below its frozen copy's for 16 proxy batches in a row (the label definition of Section 4, read causally). It cannot act before it has seen 16 degraded batches, so it is a reference, not a bound.

Each reset is audited afterwards: it is *false* if the member was not more than 1 point below its frozen copy over the 8 proxy batches before it. A false reset costs adaptation progress but, as Table 8 shows, is not necessarily harmful. The loop was checked against the earlier loops: on the real models a live run without resets equals the earlier stress run over its first 12,800 images in every sample, the frozen copies equal a standalone frozen run, and a member's first batch after a reset equals the frozen member's.

Whole-stream accuracy (Table 8, Figures 1 and 2). Without a reset the duo collapses: 13.1% on held-out and 24.2% on test, against 52.1% and 46.1% for the frozen duo (on test seed 0 the ViT-B/16 is below 5% accuracy for 42% of the stream and the ResNet-50 for 79%). Every rule rescues it, ending 3.3 to 4.4 points above the frozen duo on held-out and 4.8 to 7.7 on test.

- **The best rule is** `gap_fast_vs_slow` (56.5 held-out, 53.8 test). It beats every fixed schedule on both streams and the episodic reset on test (53.3), is 0.4 below the oracle on held-out and 1.2 above it on test.
- **The offline ranking carries to test.** The live order of the five leaders on test (53.8, 52.4, 51.8, 51.8, 50.9) is their offline order of Table 3: `gap`, `own_fast`, `own_cusum`, `own_level`, `both_fast_vs_slow`. On held-out the five are within 1.1 points (55.4–56.5) and `both_fast_vs_slow` is second.
- `both_fast_vs_slow` **fails on test.** It leaves the ResNet-50 below 5% for 32% of the stream (seed 0; the blue line of Figure 1), which the screen had flagged (it missed the ResNet-50 on test). The other four rules keep both members above 5% for essentially the whole stream (at most 1% of windows dead).

Table 8: Live runs, Vanilla DD with Tent, ViT-B/16 at $1.4{\cdot}10^{-4}$ and ResNet-50 at $5{\cdot}10^{-4}$, whole streams. Duo = mean accuracy (%) of Vanilla DD over the whole stream (mean over 2 held-out runs, 3 test seeds; the spread over runs is at most 0.5 points and reflects the data order only); Δ = gain over the frozen duo on the same images; resets per 1,000 proxy batches as ViT-B/16 / ResNet-50; false = share of resets (%) of a member that was not below its frozen copy. – = not run. Rules in the order of Table 3.

	held-out (2 runs)				test (3 seeds)			
condition	duo	Δ	resets	false	duo	Δ	resets	false
Frozen duo	52.1	–	–	–	46.1	–	–	–
No reset	13.1	-38.9	0.0 / 0.0	–	24.2	-21.9	0.0 / 0.0	–
`gap_fast_vs_slow`	56.5	+4.4	2.9 / 2.7	66	53.8	+7.7	2.0 / 3.0	91
`own_fast_vs_slow`	55.6	+3.5	2.1 / 1.4	32	52.4	+6.3	1.0 / 1.4	29
`own_cusum`	55.7	+3.6	3.0 / 1.3	44	51.8	+5.7	0.7 / 1.5	5
`own_level`	55.4	+3.3	2.7 / 1.3	36	51.8	+5.7	0.3 / 1.0	0
`both_fast_vs_slow`	55.9	+3.8	2.2 / 2.2	36	50.9	+4.8	0.8 / 1.5	55
Every 250 steps	54.3	+2.2	16.0 / 16.0	100	–	–	–	–
Every 500 steps	55.4	+3.3	8.0 / 8.0	92	51.7	+5.6	7.8 / 7.8	99
Every 1,000 steps	55.3	+3.2	3.8 / 3.8	73	52.7	+6.6	3.8 / 3.8	95
Every 2,000 steps	53.5	+1.4	1.9 / 1.9	42	–	–	–	–
Episodic (every shift)	56.4	+4.3	4.8 / 4.8	88	53.3	+7.2	4.8 / 4.8	90
Oracle (labels)	56.9	+4.8	2.1 / 2.2	0	52.6	+6.4	2.3 / 1.7	0

- **Oracle and episodic are not ceilings.** The oracle scores 52.6 on test, below `gap`, the episodic reset and the 1,000-step schedule. The episodic reset lets the ViT-B/16 collapse inside a single shift (fog) and it stays dead until the next boundary (Figure 1, middle panel).
- **Rules protect the ViT-B/16, schedules protect the ResNet-50** (Table 9). On test the ViT-B/16 scores 48.8–50.0 under the rules and 49.2 under the oracle, but 44.5–47.3 under the schedules and the episodic reset; the ResNet-50 scores 42.6–43.8 under those and 26.5–40.9 under the rules.
- **How often you reset matters.** The fixed periods score 54.3, 55.4, 55.3 and 53.5 on held-out for 250, 500, 1000 and 2000 steps: too frequent costs adaptation, too rare lets the collapse through. The held-out pick, 500, does not carry to test (51.7, against 52.7 for 1000).
- **At matched reset frequency** `gap_fast_vs_slow` (2–3 resets per 1,000 batches per member) beats the 1,000-step schedule (3.8) by 1.2 points on held-out and 1.1 on test, but only the held-out difference is outside the noise (Table 10: $+1.25 \pm 0.28$ against $+1.13 \pm 1.02$ on test, a win in 8 of 15 test visits). The own rules reset about 3 to 13 times less often than that schedule and land within it: +0.1 to +0.4 on held-out, −0.3 to −0.9 on test, all within one standard error; `both_fast_vs_slow` is −1.8 on test.
- **The audit does not predict harm.** 91% of `gap`'s test resets are false (66% on held-out), against 5% for `own_cusum`, 0% for `own_level` and 29% for `own_fast`, yet `gap` wins. Its resets alternate (Figure 3): in each test seed all 18 ViT-B/16 resets follow a ResNet-50 reset 20–69k images earlier. A plausible reading, not checked here, is that a reset member's proxy dips and recovers, which moves its partner's gap, so the rule acts partly as an emergent schedule.

What this does and does not show. At fixed stress rates, whether a reset helps depends on how often it happens as much as on whether it follows a collapse, so the comparison mixes detection with reset frequency. What the runs support: the label-free rules rescue the duo from a collapse that costs 22 points against the frozen duo on test, match or beat the best blind schedule (the `gap` rule by about 1 point, the own rules within 1 point with far fewer resets), and the offline screen predicted the live order on the test stream. They do not show that the rules detect collapses better than a schedule would at its own best rate.

8 Tuning protocol for the second stage

The live runs of Section 7 used the stress rates and compare ways of deciding a reset at a fixed collapse rate. They leave three things open: whether a reset method beats a no-reset duo that is itself tuned for continual adaptation (on held-out, the tuned no-reset Vanilla DD already scores 54.1, against 55.4–56.5 for the rules, so

Table 9: The members' own accuracy (%) over the whole stream in the live runs of Table 8; Frozen = the same members without adaptation.

	held-out		test	
condition	ViT-B/16	ResNet-50	ViT-B/16	ResNet-50
Frozen	48.8	34.9	40.4	31.7
No reset	5.1	11.6	23.8	5.8
`gap_fast_vs_slow`	53.5	43.5	50.0	40.9
`own_fast_vs_slow`	53.3	36.2	49.8	33.2
`own_cusum`	53.2	35.8	48.9	34.3
`own_level`	53.2	33.9	48.8	35.0
`both_fast_vs_slow`	53.5	38.5	49.1	26.5
Every 250 steps	50.7	42.5	–	–
Every 500 steps	51.5	43.9	44.8	42.6
Every 1,000 steps	48.6	43.9	47.3	42.6
Every 2,000 steps	39.5	42.0	–	–
Episodic (every shift)	49.0	45.7	44.5	43.8
Oracle (labels)	53.7	44.2	49.2	40.8

Table 10: Paired differences of the live runs of Table 8 in the duo's accuracy (points), $A - B$ on the same images: mean $\pm$ standard error over visits, and the number of visits in which A is better. Held-out: the visits of the two rotations pooled (32); test: the three seeds averaged per visit (15 visits). Visits are not independent (a collapse carries into the next visit; the two rotations hold the same images in another order), so the standard error is a noise floor, not a confidence interval. Computed by `scripts/41_paired_comparisons.py`.

	held-out (32 visits)		test (15 visits)	
$A - B$	diff.	wins	diff.	wins
`gap_fast_vs_slow` − every 1,000 steps	$+1.25 \pm 0.28$	24	$+1.13 \pm 1.02$	8
`gap_fast_vs_slow` − episodic	$+0.10 \pm 0.19$	12	$+0.51 \pm 1.05$	6
`gap_fast_vs_slow` − oracle	-0.41 ± 0.22	12	$+1.28 \pm 0.48$	9
`own_fast_vs_slow` − every 1,000 steps	$+0.34 \pm 0.41$	15	-0.27 ± 1.30	6
`own_cusum` − every 1,000 steps	$+0.40 \pm 0.46$	16	-0.89 ± 1.47	7
`own_fast_vs_slow` − `gap_fast_vs_slow`	-0.90 ± 0.37	9	-1.40 ± 0.55	3
Episodic − every 1,000 steps	$+1.14 \pm 0.22$	24	$+0.62 \pm 0.53$	11

the margin is 1.3–2.4 points, not the 3.3–4.4 over the frozen duo); whether the rates, thresholds and periods were chosen with the same care for every method; and whether a rule's timing adds anything to its reset frequency. This section fixes how every method is tuned and compared. The rules below were written down before any tuning run (`plan.md`) and are implemented in `src/reset/tuning.py`, so that no choice is made by eye.

What is compared, and what each has to tune (Table 11). All methods are Vanilla DD with Tent; every adapting method has two rates (the ViT-B/16's and the ResNet-50's). A reset method adds a second quantity: a rule's detection threshold (its sensitivity dial) or a schedule's period. The two interact: the rate decides how fast a member collapses, the threshold or period how often it is reset. A full grid is too expensive, so they are tuned by *alternating* search: the rates at the screened threshold, then the threshold at those rates, then the rates again one rung either side of the choice at the chosen threshold, until nothing changes. All choices use the four held-out shifts only (speckle noise, Gaussian blur, spatter, saturate; severity 5); the test stream is touched once, at the end.

Streams, rotations, runs. The tuning stream is the four held-out shifts of 50,000 images each, four rounds, round r rotated by r: 16 visits and 800,000 images. A *rotation* $k \in \{0, 1, 2, 3\}$ is the same stream opening with shift k (so the shifts follow other predecessors); rotations 0 and 2 are the *tuning* rotations, rotations 1 and 3 *validate* (nothing is chosen on them). A *run* is one pass over one stream for one configuration (one method, one pair of rates): about 70 minutes on an L40S. A run stores the per-sample correctness of the gated duo, the fixed-TS duo and both members, and every reset.

Table 11: The methods of the second stage. The gate is $\beta = 0.5$ for every method (the members and the proxy values do not depend on β, so the gate is an evaluation choice); the fixed-TS duo is scored on the same runs. The reset at shift boundaries reuses the episodic runs (script 03); only the gate is rescored.

method	tuned	members
Frozen duo	nothing	–
No reset (baseline 2)	two rates	decoupled
Reset every 500 / 1,000 steps	two rates (period fixed)	decoupled
`own_fast_vs_slow`, `own_cusum`	two rates, threshold	decoupled
`gap_fast_vs_slow`	the pair of rates, threshold	coupled
Reset at shift boundaries	done (episodic rates)	decoupled

Objective and decoupling. The score of a configuration is the whole-stream accuracy, averaged over the tuning rotations. For the decoupled methods (no reset, the schedules, the `own_*` rules) a member's trajectory depends only on its own rate and its own reset times: the gate sits outside the loop, and an `own` rule reads only that member's proxy. One run therefore measures *both* members at their rates, whatever the partner's rate is, and each rate is chosen on its member's own accuracy, in two independent ladders; the chosen pair is then run once as a pair and the gated duo reported. For the coupled rule (`gap_*`: the ViT-B/16's resets depend on the ResNet-50's proxy) the pair of rates is one point of a plane and is scored by the gated duo. Thresholds are always scored by the gated duo.

Ladders. Rates live on the $\sqrt{2}$ grid of the earlier tuner, $\text{rate}(k) = a \cdot 2^{k/2}$ with anchor $a = 7{\cdot}10^{-5}$ (ViT-B/16) and $5{\cdot}10^{-4}$ (ResNet-50), rounded to three digits. The first ladder uses every second rung (steps of 2): ViT-B/16 $3.5 \cdot 10^{-5}, 7 \cdot 10^{-5}, 1.4 \cdot 10^{-4}, 2.8 \cdot 10^{-4}$ and ResNet-50 $1.25 \cdot 10^{-4}, 2.5 \cdot 10^{-4}, 5 \cdot 10^{-4}, 10^{-3}$, centred on the stress pair, and is run on rotation 0. A run pairs the two ladders rung by rung (that is why the decoupling matters: four runs give eight points).

Selection rules (fixed in advance).

1. **Refine.** The best rung of rotation 0 must have both its $\sqrt{2}$ neighbours tried; if not, they are run and the best is recomputed, until it has. The best rung is then a local maximum of what was tried ("interior").
2. **Second rotation.** The best rung and its two neighbours are run on rotation 2, and the choice among the three is made on the mean of the two rotations (visits pooled, 32).
3. **Ties.** Among the candidates within one *paired* standard error of the best (the standard error of the per-visit differences to the best, so the shared difficulty of each visit cancels), take the one with the fewest resets per 1,000 proxy batches, then the lowest rate. Without this rule noise would pick the most aggressive rate among equals.
4. **No boundary, and a cap.** The chosen value must have a tried value on each side. If the tie-break pushed it to an end of the tried range, the next rung beyond is run on both tuning rotations and the choice is made again. A ladder is extended by at most two $\times 2$ steps (four $\sqrt{2}$ rungs) beyond its starting ladder on each side; the cap bounds rule 1 as well, and a value still at the capped end is reported as a plateau (a quantity that does not matter) instead of being chased. *Correction made during the runs:* the first implementation enforced the cap only after a choice, so rule 1 would have kept running the neighbour of an argmax that crept down by hundredths per rung. This appeared on the threshold of `own_cusum` (Section 9), where the rung beyond the cap ($h = 0.2$) was about to be requested after $h = 0.25$ scored 57.84 against 57.83 at $h = 0.35$. The code was changed to apply the written rule (`src/reset/tuning.py`, with a self-test of a creeping ladder); every selection made before the change was recomputed and is unchanged.
5. **Guard.** A configuration in which the member is *dead* (four consecutive windows of 5,120 images below 5% accuracy) on a tuning rotation is excluded.
6. **Coupled rule.** A cell is a pair (ViT-B/16 rung, ResNet-50 rung). Start on the diagonal of the two ladders; the best cell of rotation 0 must have its neighbourhood tried: the four axis neighbours (one member's rung up or down) and the two cells that scale both members together (the direction of the starting ladder), six runs; the best cell and its two best neighbours go to rotation 2; the chosen cell must again have its neighbourhood tried. The two off-diagonal corners of the full 3×3 block are not run.

Table 12: Threshold ladders of the three leaders, other parameters as selected at budget 1. Calibration pool (S+H): false alarms per 1,000 proxy batches, miss rate, mean delay (256-image batches, a miss counts its whole stretch); test streams: false alarms. Bold: the setting with the shortest delay within 1 false alarm per 1,000 on the calibration pool. `scripts/37c_threshold_ladder.py`.

rule	threshold	FA	miss	delay	test FA
`gap_fast_vs_slow`	0.35	2.92	0.27	74.0	2.74
(δ)	0.5	2.14	0.23	108.8	2.47
	0.7	1.57	0.19	83.7	1.83
	1.0	**0.91**	**0.35**	**109.6**	**1.19**
	1.25	0.35	0.54	177.2	0.64
	1.5	0.09	0.54	194.8	0.27
`own_fast_vs_slow`	0.35	1.38	0.27	104.5	1.64
(δ)	0.5	1.19	0.27	108.1	1.46
	0.7	**0.97**	**0.27**	**103.9**	**0.82**
	1.0	0.69	0.31	112.8	0.27
	1.25	0.31	0.38	131.1	0.00
	1.5	0.00	0.46	232.4	0.00
`own_cusum`	0.5	1.57	0.31	124.9	0.00
(h)	1	0.82	0.31	137.0	0.00
	2	**0.72**	**0.31**	**133.2**	**0.00**
	4	0.63	0.31	145.1	0.00
	8	0.63	0.31	147.8	0.00

The no-reset baseline. Its ladder already exists: the continual tuner (04b) ran 15 rungs per member on this very stream, whole-stream accuracy per rung. The same rules are applied to it (with rotation 2 added for the best rung and its neighbours), so the baseline is tuned by the same objective as the others. The 04b pair of "conservative" rates (one rung below the largest rate that stays stable) is reported next to it, because it is the safe choice a practitioner without a held-out stream would make.

Thresholds. The threshold ladder is the $\sqrt{2}$ grid anchored at the screened value (δ of `fast_vs_slow`, h of `cusum`); the other parameters (the two averaging speeds, ν, the warm-up W_0) stay as the screen selected them. It is run at the tuned rates with the same rules, scored by the gated duo; then the rates are rechecked one rung either side (alternating search).

The screen's thresholds sat at the edge of its grid. The first screen chose, at budget 1, the *smallest* threshold of its grid for all three leaders ($\delta = 1.0$ for the two `fast_vs_slow` rules, $h = 2.0$ for `cusum`), so nothing said that a more sensitive threshold was not better. Table 12 extends the sweep down (calibration pool S+H and, frozen, test T; the other parameters as selected). For `gap_fast_vs_slow` and `own_cusum` the grid edge was harmless: the next lower threshold exceeds the budget (1.57 and 1.57 false alarms per 1,000) without a shorter delay. For `own_fast_vs_slow` it was not: $\delta = 0.7$ stays within the budget (0.97) and has a shorter delay (104 against 113 batches), so the starting threshold of its tuning is 0.7, not 1.0. The calibration false-alarm rate grows steadily as the threshold falls (to 1.4–3.5 per 1,000 at the lowest values), so the ladders bracket the budget on both sides.

Controls and the final test. Detection has to beat frequency. For the best rule a *random* trigger resets each member the same number of times as the rule did on the same stream, at random times (five draws); if the rule does not beat it, what the rule provides is reset frequency, not detection. A *staggered* schedule (both members on the same period, the ViT-B/16 half a period later) is run only if the `gap` rule wins, to test whether its alternation alone explains it. The final comparison runs every tuned configuration once on the test stream (15 corruptions, one pass) in five orderings of the 15 shifts, with the gated duo ($\beta = 0.5$) and the fixed-TS duo scored on each run, and paired differences over the shifts. Three claims are fixed in advance: (a) the best rule beats the tuned no-reset baseline by more than the paired standard error; (b) it beats the random control at

Rate	Every 500 steps	Every 1,000 steps	own fast/slow (δ=0.7)	own cusum (h=2)	own cusum (h=0.25)
3.5×10^{-5}	49.83	50.38	50.87 / 50.93	50.58	–
4.95×10^{-5}	–	50.88	–	–	–
7×10^{-5}	50.65	**51.47 / 51.29**	52.12	51.90	–
9.9×10^{-5}	51.17 / 51.18	51.74 / 50.82	–	–	–
1.4×10^{-4}	**51.48 / 51.56**	49.33 / 47.87	53.21	53.31	–
1.98×10^{-4}	50.43 / 50.73	–	53.74	54.00 / 53.97	54.13 / 54.08
2.8×10^{-4}	48.29	42.30†	54.36	**54.55 / 54.55**	**54.73 / 54.65**
3.96×10^{-4}	–	–	54.45 / 54.57	54.48 / 54.35	54.56 / 54.37
5.6×10^{-4}	–	–	**54.81 / 54.79**	–	–
7.92×10^{-4}	–	–	54.50 / 54.72	–	–

Table 13: ViT-B/16's own accuracy (%, whole held-out stream) at each rung of its learning-rate ladder, under each method with the trigger active: rotation 0 / rotation 2 (rotation 2 only for the argmax and its neighbours). Bold: the chosen rung (tie-break included); †: the member died. Produced by `scripts/43_tuning_tables.py`.

matched reset counts; (c) the comparison with the boundary reset and the tuned schedules is reported whichever way it goes.

9 Results of the tuning

Everything in this section is held-out: the four shifts, rotations 0 and 2 to choose, rotations 1 and 3 only to validate. The tuning took about 115 runs of 68 minutes (the rotation-0 ladders, rotation 2 for each argmax and its neighbours, the extensions, the confirmations and the controls); the tables are generated from the stored runs by `scripts/43_tuning_tables.py`. Throughout, the number to compare with is the tuned no-reset duo, 55.5 (Table 18), not the 54.1 quoted earlier: that was the gate at $\beta = 2$; at $\beta = 0.5$ the same no-reset members score 55.5, so the room a reset method has is about a point smaller than Section 8 first assumed.

Rates under the active trigger (Tables 13–16).

- **Without resets the two members tolerate very little.** The accuracy-tuned rates are $1.75 \cdot 10^{-5}$ (ViT-B/16, 52.0) and $3.1 \cdot 10^{-5}$ (ResNet-50, 41.5), only 1.4 times the conservative pair of 04b; two rungs above, at $3.5 \cdot 10^{-5}$, the ViT-B/16 is at 45 and at $4.9 \cdot 10^{-5}$ at 32 (Table 16). The tuned no-reset duo gains 0.4 over the conservative one (55.5 against 55.1), a gain of the same order as the differences among the methods below.
- **A schedule cannot improve the ViT-B/16.** Its best rates ($1.4{\cdot}10^{-4}$ every 500 steps, $7{\cdot}10^{-5}$ every 1,000) give 51.5 and 51.4, below the tuned no-reset ViT-B/16 (52.0): a periodic reset removes a healthy ViT-B/16's progress as often as a collapsing one's. The schedules lift only the ResNet-50 (44.3 and 43.8, against 41.5 without resets), consistent with a slow decay that a periodic reset caps, and that is where their duo gain comes from.
- **The `own` rules let the ViT-B/16 run at a much higher rate.** Because the reset fires when the ViT-B/16's own proxy drops, the ViT-B/16 can adapt at 2.8–$5.6 \cdot 10^{-4}$, 16 to 32 times the no-reset optimum and 2 to 4 times a schedule's, and its accuracy rises to 54.5–54.8 (against 52.0). The optimum is interior: `own_fast_vs_slow` is best at $5.6 \cdot 10^{-4}$ (54.8; 54.5 at $7.9 \cdot 10^{-4}$) after its ladder had been extended three rungs above the original top ($2.8 \cdot 10^{-4}$), `own_cusum` at $2.8 \cdot 10^{-4}$ (54.6; 54.4 at $4 \cdot 10^{-4}$).
- **They do nothing for the ResNet-50.** Its best rates are the low ones ($8.8 \cdot 10^{-5}$ for `own_fast_vs_slow`, 41.1; 10^{-3} for `own_cusum`, 39.8 and 41.0 on the two rotations, on a ragged curve: 34 to 40 across its rungs), no better than the 41.5 without resets. The screen had warned of it: the ResNet-50 decays like a ramp that the fast/slow statistic detects late (delay about 150 batches against 11–32 for the ViT-B/16).
- **The coupled rule helps both members a little.** The pair ($1.4{\cdot}10^{-4}, 5{\cdot}10^{-4}$) is the chosen cell (Table 15); it is tied within one paired standard error with two neighbours, and the tie-break takes the lowest rates. Its ViT-B/16 (53.6) is below the `own` rules' and its ResNet-50 (43.7) above them.

Per-member detector settings for the ResNet-50 (offline screen; no live run). The tuning above gave both members the same detector setting. A per-member screen (`scripts/44_screen_member_detector.py`;

Rate	Every 500 steps	Every 1,000 steps	own fast/slow (δ=0.7)	own cusum (h=2)	own cusum (h=0.25)
6.25×10^{-5}	–	–	40.73 / 40.87	–	–
8.84×10^{-5}	–	–	**41.11 / 41.21**	–	–
1.25×10^{-4}	40.41	41.63	41.31 / 41.20	38.70	–
1.77×10^{-4}	–	–	41.20 / 40.77	–	–
2.5×10^{-4}	42.35	43.13	40.04	34.89	–
3.54×10^{-4}	–	43.60 / 43.74	–	–	–
5×10^{-4}	43.89	**43.75 / 43.96**	36.74	35.96	–
7.07×10^{-4}	**44.28 / 44.29**	43.29 / 43.55	–	33.73 / 34.26	34.27 / 34.61
1×10^{-3}	44.30 / 44.28	41.84	39.33	**39.76 / 41.04**	**41.22 / 42.18**
1.41×10^{-3}	43.54 / 43.69	–	–	37.22 / 36.33	38.91 / 37.82
2×10^{-3}	–	–	38.46 / 36.85	–	–

Table 14: ResNet-50's own accuracy (%, whole held-out stream) at each rung of its learning-rate ladder, under each method with the trigger active: rotation 0 / rotation 2 (rotation 2 only for the argmax and its neighbours). Bold: the chosen rung (tie-break included); †: the member died. Produced by `scripts/43_tuning_tables.py`.

ViT rate	ResNet rate	Duo, rotation 0	Duo, rotation 2	Resets/1,000 (rot 0)
3.5×10^{-5}	1.25×10^{-4}	54.46	–	3.2 / 3.8
7×10^{-5}	2.5×10^{-4}	56.24	–	1.9 / 1.9
9.9×10^{-5}	3.54×10^{-4}	56.65	–	2.2 / 2.6
9.9×10^{-5}	5×10^{-4}	56.60	–	2.2 / 2.2
9.9×10^{-5}	7.07×10^{-4}	56.57	–	2.2 / 2.6
1.4×10^{-4}	3.54×10^{-4}	57.05	–	3.2 / 3.5
1.4×10^{-4}	5×10^{-4}	57.31	**57.02**	2.9 / 2.6
1.4×10^{-4}	7.07×10^{-4}	57.42	57.10	2.9 / 2.9
1.4×10^{-4}	1×10^{-3}	56.93	–	1.9 / 3.5
1.98×10^{-4}	5×10^{-4}	57.31	–	3.5 / 2.2
1.98×10^{-4}	7.07×10^{-4}	57.15	56.88	3.5 / 2.6
1.98×10^{-4}	1×10^{-3}	57.00†	–	2.6 / 3.2
2.8×10^{-4}	1×10^{-3}	56.92†	–	3.5 / 3.2

Table 15: The coupled rule (gap fast/slow, $\delta = 1$): the gated duo's accuracy (%, $\beta = 0.5$) at each (ViT, ResNet) pair tried, since its ViT resets depend on the ResNet's proxy and the members cannot be tuned separately. Bold: the chosen pair; †: a member died.

calibration streams only, the ResNet-50's 18 degraded stretches only, false alarms per 1,000 ResNet-50 batches) tried ten averaging-speed pairs, three warm-ups and ten thresholds of `fast_vs_slow` on the ResNet-50. The rule, written down before the run (`plan.md`), was to go on to live runs only if the best setting cut the ResNet-50's delay at budget 1 by at least 20% against the shared setting. It cut it by 12% (131 against 149 batches, with $(\alpha_f, \alpha_s) = (0.02, 0.005)$, $W_0 = 100$, $\delta = 0.7$ against $(0.05, 0.005)$, 20, 1), which 18 events cannot separate from noise, so no live run was made. The limit is not the parameters, nor the family: on the ResNet-50 the best delays at the same budget are 149 (`fast_vs_slow`), 183 (`cusum`), 202 (`level`), 283 (`imm_vs_ema`) and 300 (`slope`) batches, against 19–24 for all five on the ViT-B/16. Seven of the ResNet-50's 18 degraded stretches are dips of 16–97 batches whose proxy never moves; in 3 of the 11 long ones the proxy sits above its pre-onset level on average while the accuracy falls, and in 6 of them it falls only 110–391 batches after the onset. The ResNet-50's signal lags or misleads, so a detector tuned on it cannot be early; a better ResNet-50 trigger needs another signal for the ResNet-50 (another proxy), not other averaging speeds. The replay measures only the first collapse of each member (the trajectory is not changed), so a live run could still differ.

Thresholds (Table 17). Thresholds matter much less than rates. `gap_fast_vs_slow` has a clear interior optimum at $\delta = 1$ (56.2, 56.8, 57.3, 56.3 for $\delta = 0.5, 0.7, 1, 1.4$ on rotation 0, confirmed on rotation 2): below it the rule resets too often, above it too late, and its rates, tuned at $\delta = 1$, need no recheck. `own_fast_vs_slow` is flat between $\delta = 0.35$ and 0.7 (57.8–57.9) and loses 0.15 at 1; the choice, 0.7, is the base of the ladder and interior. `own_cusum` is a plateau: its duo accuracy creeps from 57.44 ($h = 2.8$) to 57.84 ($h = 0.25$), the number

Member	Rate	Own accuracy, rotation 0 / rotation 2
ViT	2.2×10^{-6}	49.60
ViT	3.1×10^{-6}	49.86
ViT	4.4×10^{-6}	50.20
ViT	6.2×10^{-6}	50.62
ViT	8.75×10^{-6}	51.07
ViT	1.26×10^{-5}	51.62 / 51.62
ViT	1.75×10^{-5}	**52.02 / 51.97**
ViT	2.45×10^{-5}	51.47 / 50.09
ViT	3.5×10^{-5}	45.25
ViT	4.9×10^{-5}	31.64
ViT	7×10^{-5}	43.08
ViT	9.8×10^{-5}	36.68
ViT	1.4×10^{-4}	40.42
ViT	1.96×10^{-4}	39.50
ViT	2.8×10^{-4}	38.20
ResNet	1.6×10^{-5}	40.79
ResNet	2.2×10^{-5}	41.21
ResNet	3.1×10^{-5}	**41.51 / 41.56**
ResNet	4.4×10^{-5}	41.60 / 41.62
ResNet	6.25×10^{-5}	41.33 / 41.33
ResNet	9×10^{-5}	40.57
ResNet	1.25×10^{-4}	38.26
ResNet	1.75×10^{-4}	32.52
ResNet	2.5×10^{-4}	30.13
ResNet	3.5×10^{-4}	23.55
ResNet	5×10^{-4}	23.69
ResNet	7×10^{-4}	21.84
ResNet	1×10^{-3}	19.86
ResNet	1.4×10^{-3}	18.07
ResNet	2×10^{-3}	16.09

Table 16: No reset: the ladder of the continual-TTA tuner (04b), each member scored on its own accuracy; the argmax and its neighbours were also run on rotation 2. Bold: the accuracy-tuned rate. The conservative pair of 04b (1.26×10^{-5} / 2.2×10^{-5}) is a stability rule, reported beside it.

of resets is the same at every h from 0.5 to 2.8 (4.8 per 1,000 batches for the ViT-B/16, 3.8 for the ResNet-50), and only below about 0.35 does the ViT-B/16 reset a little more (5.1–5.4). That is what one expects when a collapse is abrupt: the statistic crosses every threshold in the same batch, and a threshold only matters once noise can cross it. The ladder reached the cap of two extensions at $h = 0.25$, so the threshold is reported as a plateau at the capped end, not as a located optimum. Because h changed from its base (2), its rates were rechecked one rung either side at $h = 0.25$ (alternating search; Section 8, last column of Tables 13 and 14): the ViT-B/16 stays at $2.8 \cdot 10^{-4}$ (54.7 on the two tuning rotations; 54.5 at $4 \cdot 10^{-4}$ and 54.1 at $2 \cdot 10^{-4}$) and the ResNet-50 at 10^{-3} (41.7; 34.4 at $7.1 \cdot 10^{-4}$ and 38.4 at $1.4 \cdot 10^{-3}$). Nothing changed, so the alternating search stopped, and `own_cusum` is final at $h = 0.25$ with the pair $(2.8 \cdot 10^{-4}, 10^{-3})$, confirmed on all four rotations (Table 18).

The tuned configurations (Table 18). On all four rotations the ordering is the same and the validation rotations agree with the tuning ones to within 0.3 (0.6 for the gap rule on rotation 1). Over the tuned no-reset duo (55.5) the three rules gain 1.5 to 2.4 points: `own_fast_vs_slow` (57.8–57.9) and `own_cusum` (57.6–57.9) about 2.3, `gap_fast_vs_slow` (56.7–57.3) 1.5; the schedules gain 0.7 (every 500 steps) and 0.4 (every 1,000). Paired over the 32 visits of the two validation rotations (nothing was chosen on them; visits are correlated, so the errors are a noise floor): `own_fast_vs_slow` $+2.3 \pm 0.6$, `own_cusum` $+2.2 \pm 0.5$, `gap_fast_vs_slow` $+1.4 \pm 0.3$, every 500 steps $+0.7 \pm 0.2$, every 1,000 steps $+0.4 \pm 0.2$; the accuracy-tuned no-reset rates over the conservative ones $+0.39 \pm 0.08$. Among the rules, `own_fast_vs_slow` and `own_cusum` are indistinguishable ($+0.16 \pm 0.29$) and both lead `gap_fast_vs_slow` by about two standard errors ($+0.9 \pm 0.4$ and $+0.7 \pm 0.3$); `own_fast_vs_slow`

Rule	Threshold	Duo, rotation 0	Duo, rotation 2	Resets/1,000 (ViT / ResNet)
own fast/slow	0.35	57.81	–	7.0 / 1.9
own fast/slow	0.5	57.88	57.76	6.7 / 1.0
own fast/slow	0.7	**57.88**	**57.84**	6.7 / 1.0
own fast/slow	1	57.67	57.78	6.1 / 0.6
own cusum	0.25	**57.84**	**57.90**	5.4 / 3.8
own cusum	0.35	57.83	57.89	5.1 / 3.8
own cusum	0.5	57.66	–	4.8 / 3.8
own cusum	0.7	57.63	–	4.8 / 3.8
own cusum	1	57.62	–	4.8 / 3.8
own cusum	1.4	57.48	–	4.8 / 3.8
own cusum	2	57.47	57.73	4.8 / 3.8
own cusum	2.8	57.44	–	4.8 / 3.8
gap fast/slow	0.5	56.23	–	6.7 / 6.7
gap fast/slow	0.7	56.76	56.51	4.5 / 4.8
gap fast/slow	1	**57.31**	**57.02**	2.9 / 2.6
gap fast/slow	1.4	56.30	56.40	1.9 / 1.3

Table 17: Stage 3: the detection threshold at the tuned rates (gated duo, $\beta = 0.5$, whole held-out stream). Rotation 2 only for the argmax and its neighbours. Bold: the chosen threshold.

Method	ViT rate	ResNet rate	Rot 0	Rot 1*	Rot 2	Rot 3*	Fixed-TS (mean)
No reset, accuracy-tuned rates	1.75×10^{-5}	3.1×10^{-5}	55.5	55.5	55.5	55.5	55.8
No reset, conservative rates	1.26×10^{-5}	2.2×10^{-5}	55.1	55.1	55.1	55.2	55.4
Every 500 steps	1.4×10^{-4}	7.07×10^{-4}	56.2	56.2	56.2	56.2	56.2
Every 1,000 steps	7×10^{-5}	5×10^{-4}	55.9	56.0	55.9	55.9	56.0
Own fast/slow trigger	5.6×10^{-4}	8.84×10^{-5}	57.9	57.8	57.8	57.9	57.9
Gap fast/slow trigger	1.4×10^{-4}	5×10^{-4}	57.3	56.7	57.0	57.1	57.1
Own CUSUM trigger	2.8×10^{-4}	1×10^{-3}	57.8	57.8	57.9	57.6	57.2

Table 18: The tuned configurations on all four rotations of the held-out stream (gated duo, $\beta = 0.5$, whole-stream accuracy in %). Rotations 0 and 2 were used for the tuning; * rotations 1 and 3 only validate.

leads the 500-step schedule by $+1.6 \pm 0.6$.

Control: does the timing matter? (Table 19). For the rule that leads on validation, `own_fast_vs_slow`, five random triggers reset each member as many times as the rule did, at random steps, on validation rotation 1. Every draw is beaten: 52.9 to 55.6 against 57.8, differences of +2.2 to +4.9 points with paired standard errors of 0.7–1.5, and the rule wins 9 to 13 of the 16 visits. The advantage is almost entirely the ViT-B/16 (+9 to +19 points; the ViT-B/16 is 35–46 under random resets, presumably because a reset that misses the collapse leaves it collapsed for a long stretch, which the traces of the runs would show). The ResNet-50 is indistinguishable from random (−0.7 to +0.1, standard errors about 0.5): at about one reset per 1,000 batches the rule does for the ResNet-50 nothing a random reset would not. So on held-out the *detection* adds to reset frequency for the ViT-B/16, and not for the ResNet-50.

10 Final test

Every configuration of Section 9 was frozen in `out/results/reset_tune/final_configs.json` before the first test run and is run once on the test stream: the 15 corruptions at severity 5, one pass of 750,000 images, in five orderings (the standard order and CoTTA's random orders 0–3), with the gate at $\beta = 0.5$ and the fixed-TS duo scored on the same runs (`scripts/42_final_test.py`). The control is one random draw per ordering, resetting each member as often as `own_fast_vs_slow` did on the same ordering. The frozen duo and the reset at every shift are the episodic runs of script 03 (order-independent). Paired differences are taken per corruption, each corruption's accuracy averaged over the orderings, with a bootstrap over the 15 corruptions. The orderings

	Duo	ViT-B/16	ResNet-50	Resets/1,000	Duo: rule − random	ViT-B/16	ResNet-50
The rule itself	57.8	54.5	41.1	6.7 / 0.6	–	–	–
Random, draw 0	55.0	43.6	41.8	6.7 / 0.6	+2.84±1.08 (9/16)	+10.9±4.0	-0.69±0.55
Random, draw 1	54.0	41.9	41.2	6.7 / 0.6	+3.78±1.19 (12/16)	+12.5±3.9	-0.06±0.63
Random, draw 2	52.9	35.4	41.0	6.7 / 0.6	+4.93±1.29 (13/16)	+19.1±5.3	+0.10±0.55
Random, draw 3	53.0	39.0	41.2	6.7 / 0.6	+4.77±1.54 (12/16)	+15.5±4.9	-0.03±0.57
Random, draw 4	55.6	45.5	41.5	6.7 / 0.6	+2.15±0.69 (13/16)	+9.0±2.9	-0.34±0.38
Mean of the draws	54.1	41.1	41.3	–	+3.69	+13.4	-0.20

Table 19: Stage 4, held-out validation rotation 1: `own_fast_vs_slow` ($\delta = 0.7$; ViT-B/16 $5.6{\cdot}10^{-4}$, ResNet-50 $8.84{\cdot}10^{-5}$) against a trigger that resets each member as often as the rule did, at random steps (five draws). Accuracy (%) and the paired difference per visit (mean±SE over the 16 visits; visits won by the rule). The rule's advantage is almost all in the ViT-B/16.

Method	Vanilla DD	Fixed-TS duo	ViT	ResNet	Resets/1,000
Frozen (no adaptation)	46.1	46.1	40.4	31.7	–
Reset at every shift (episodic)	52.2	–	42.9	43.9	–
No reset, accuracy-tuned rates	50.2±0.2	50.4±0.2	45.1±0.3	38.4±0.3	0.0 / 0.0
No reset, conservative rates	49.8±0.2	50.0±0.1	44.4±0.2	38.3±0.3	0.0 / 0.0
Every 500 steps	52.0±0.1	51.9±0.1	43.8±0.2	42.5±0.1	7.9 / 7.9
Every 1,000 steps	51.7±0.1	51.5±0.1	43.8±0.4	42.0±0.2	3.8 / 3.8
Own fast/slow trigger	54.0±0.3	54.2±0.3	50.2±0.8	38.1±0.5	3.7 / 0.7
Gap fast/slow trigger	53.1±0.3	53.1±0.3	47.7±0.6	40.7±0.4	2.6 / 3.4
Own CUSUM trigger	53.4±0.3	53.0±0.3	49.5±0.5	37.0±0.3	2.0 / 3.8
Random resets, matched to Own fast/slow trigger	52.6±0.5	48.5±1.2	44.3±1.3	39.0±0.6	3.7 / 0.7

Table 20: Final test: accuracy (%, mean over the 15 corruptions) on the ImageNet-C test stream, every configuration as tuned on the held-out stream, the gate at $\beta = 0.5$. Mean±SE over 5 orderings of the corruptions (standard order and CoTTA's random orders); resets per 1,000 proxy batches for the ViT / ResNet. Frozen and the episodic reset are order-independent (scripts/03b rows). Produced by `scripts/42_final_test.py`.

reuse the same images, so they are not independent replicates; the standard errors over orderings in Table 20 are small for that reason.

Claim (a): the best rule beats the tuned no-reset duo — it does, for all three rules. Tuned without resets the duo scores 50.2 (conservative rates 49.8: $+0.45 \pm 0.15$ for the tuning, 12 of 15 corruptions), 4.1 points above the frozen duo. `own_fast_vs_slow` adds $+3.8 \pm 1.0$ (bootstrap interval $[+2.1, +5.9]$; 14 of 15 corruptions; positive in each of the five orderings, +2.8 to +4.4), `own_cusum` $+3.2 \pm 0.9$ ($[+1.6, +5.0]$; 14 of 15) and `gap_fast_vs_slow` $+2.8 \pm 0.5$ ($[+2.0, +3.8]$; 15 of 15). The two schedules add +1.8 and +1.5. The held-out ordering of the rules holds (`own_fast` 54.0, `own_cusum` 53.4, `gap` 53.1), but on test the differences among the three rules are inside the noise: `own_fast` over `gap` $+0.9 \pm 0.6$ ($[-0.2, +2.3]$), over `own_cusum` $+0.6 \pm 0.4$ ($[-0.2, +1.3]$), and `own_cusum` over `gap` $+0.3 \pm 0.6$. The margin over the tuned no-reset duo is larger on test (3.8) than on held-out (2.3).

Claim (b): the rule beats random resets at matched counts — on average, and less than on held-out. `own_fast_vs_slow` scores 54.0 against 52.6 for the random trigger with the same reset counts: $+1.4 \pm 0.5$ ($[+0.5, +2.3]$; 12 of 15 corruptions). Per ordering the difference is +0.7, +1.8, +3.6, +1.6 and −0.7: in one ordering the random draw wins. With one draw per ordering the control is itself noisy (on held-out the five draws spread over 52.9–55.6), and the held-out margin (about +3.7 on average over five draws, on a single rotation) was larger than the test margin. The random control also depends on the gate: its fixed-TS duo scores 48.5, 4.1 points below its gated duo, whereas the rule's two duos agree (54.2 and 54.0). The control was run for `own_fast_vs_slow` only; for `own_cusum` and `gap` the claim is untested.

Difference (Vanilla DD, points)	Mean	SE	95% CI	Corruptions won
Every 500 steps − No reset, accuracy-tuned rates	+1.81	0.44	[+0.95, +2.60]	14/15
Every 1,000 steps − No reset, accuracy-tuned rates	+1.48	0.48	[+0.51, +2.36]	13/15
Own fast/slow trigger − No reset, accuracy-tuned rates	+3.78	1.02	[+2.08, +5.90]	14/15
Gap fast/slow trigger − No reset, accuracy-tuned rates	+2.84	0.47	[+1.98, +3.79]	15/15
Own CUSUM trigger − No reset, accuracy-tuned rates	+3.16	0.89	[+1.63, +5.01]	14/15
Own fast/slow trigger − Random resets, matched to Own fast/slow trigger	+1.39	0.46	[+0.54, +2.32]	12/15
Own fast/slow trigger − Reset at every shift (episodic)	+1.80	1.41	[-0.39, +4.86]	8/15
Own fast/slow trigger − Every 500 steps	+1.97	1.30	[-0.12, +4.76]	10/15
Own fast/slow trigger − Every 1,000 steps	+2.30	1.35	[+0.09, +5.18]	10/15
Gap fast/slow trigger − Reset at every shift (episodic)	+0.86	0.82	[-0.37, +2.65]	8/15
Gap fast/slow trigger − Every 500 steps	+1.03	0.70	[-0.08, +2.54]	9/15
Gap fast/slow trigger − Every 1,000 steps	+1.37	0.75	[+0.13, +2.95]	11/15
Own CUSUM trigger − Reset at every shift (episodic)	+1.17	1.29	[-0.86, +3.93]	7/15
Own CUSUM trigger − Every 500 steps	+1.35	1.20	[-0.64, +3.85]	9/15
Own CUSUM trigger − Every 1,000 steps	+1.68	1.24	[-0.38, +4.28]	10/15
Own fast/slow trigger − Gap fast/slow trigger	+0.94	0.64	[-0.17, +2.25]	10/15
Own fast/slow trigger − Own CUSUM trigger	+0.62	0.39	[-0.16, +1.33]	11/15
Gap fast/slow trigger − Own CUSUM trigger	-0.32	0.59	[-1.45, +0.79]	9/15
No reset, accuracy-tuned rates − Frozen (no adaptation)	+4.11	0.86	[+2.41, +5.66]	13/15
No reset, accuracy-tuned rates − No reset, conservative rates	+0.45	0.15	[+0.17, +0.72]	12/15

Table 21: Paired differences on the test stream: per corruption (its accuracy averaged over the orderings), mean, standard error over the 15 corruptions, bootstrap 95% interval over corruptions (10,000 resamples) and the number of corruptions the first method wins. Corruptions are not independent, so the interval is a noise floor, not a guarantee.

Claim (c): against the boundary reset and the schedules — reported as it came out. No rule is separated from the reset at every shift, which is told where the shifts are: `own_fast_vs_slow` $+1.8 \pm 1.4$ ($[-0.4, +4.9]$), `own_cusum` $+1.2 \pm 1.3$ ($[-0.9, +3.9]$), `gap_fast_vs_slow` $+0.9 \pm 0.8$ ($[-0.4, +2.7]$), 7 to 8 of 15 corruptions each. Against the tuned schedules the rules are ahead by about +1 to +2.3 but the intervals mostly include zero: against the 1,000-step schedule `own_fast` ($[+0.1, +5.2]$) and `gap` ($[+0.1, +3.0]$) just exclude it, `own_cusum` ($[-0.4, +4.3]$) does not; against the 500-step schedule none does. The `own_fast`–schedule difference is positive in all five orderings, which counts for little because the orderings share their images.

Where the gain comes from. As on held-out, it is the ViT-B/16 (Table 20): 50.2 (`own_fast`), 49.5 (`own_cusum`) and 47.7 (`gap`) against 45.1 without resets and 43.8 for the schedules. The ResNet-50 gain belongs to the schedules (42.0–42.5), not to the rules (38.1 for `own_fast`, 37.0 for `own_cusum`, 40.7 for `gap`); the two `own` rules leave it no better than without resets (38.4), the same result as on held-out. Table 22 shows that the gain is uneven: contrast alone is +15.4 points for `own_fast` over no reset (54.7 against 39.3; the schedules lose 2–3 points there), which is about 1.0 of its 3.8-point average gain. Without contrast `own_fast` is still $+2.95 \pm 0.63$ over the tuned no-reset duo (13 of 14 corruptions), `own_cusum` $+2.5 \pm 0.6$ and `gap` $+2.5 \pm 0.4$; but the lead over random shrinks to $+1.2 \pm 0.5$, over the 1,000-step schedule to $+1.2 \pm 0.8$ and over the boundary reset to $+0.5 \pm 0.7$. This leave-one-corruption-out is a description of the table, not a new test. The schedules are ahead of the rules on a few corruptions where the ViT-B/16 does not collapse and a reset only costs (zoom blur 51–52 against 49 and 47, snow 54 against 50–53, fog 63 against 60–62).

What the test supports. A label-free reset rule, with rates and threshold tuned on four held-out shifts under fixed rules, beats an equally tuned no-reset duo by 2.8 to 3.8 points on the test stream, with bootstrap intervals that exclude zero; for `own_fast_vs_slow` part of this is timing, not frequency (+1.4 over random resets at the same counts). It does not beat the episodic reset that is told the shifts, it is not separated from the tuned schedules by a margin larger than the noise, and the three rules cannot be told apart on test. The effect is carried by the ViT-B/16 and by a few corruptions, most of all contrast.

Method	Gauss	Shot	Impul	Defoc	Glass	Motn	Zoom	Snow	Frost	Fog	Brit	Contr	Elast	Pixel	JPEG	Avg.
No reset, accuracy-tuned rates	42.9	43.7	46.2	39.2	35.8	48.1	47.7	50.7	56.2	59.1	73.6	39.3	54.4	57.2	59.3	50.2
No reset, conservative rates	42.3	42.8	45.6	38.4	34.6	47.3	47.4	50.5	56.2	59.8	73.7	37.7	54.2	57.0	59.0	49.8
Every 500 steps	45.0	46.1	46.5	40.4	37.4	49.3	**51.5**	54.1	56.5	63.2	74.6	37.0	57.0	60.9	60.9	52.0
Every 1,000 steps	44.1	46.2	45.7	39.7	36.6	49.4	51.1	54.4	56.4	**63.3**	74.4	36.2	56.7	60.4	60.9	51.7
Own fast/slow trigger	45.1	48.2	46.9	**45.4**	42.3	**51.8**	49.0	50.0	57.3	61.8	74.0	**54.7**	56.5	63.9	**63.3**	54.0
Gap fast/slow trigger	**45.5**	**48.3**	46.4	42.6	40.2	50.1	51.4	52.1	57.0	61.8	74.4	46.6	56.2	61.3	62.0	53.1
Own CUSUM trigger	45.1	47.6	46.3	45.0	43.2	51.4	46.7	53.1	**57.9**	60.4	73.9	52.0	55.2	61.1	61.9	53.4
Random resets, matched to Own fast/slow trigger	43.9	46.7	46.4	45.1	**43.5**	48.4	49.0	45.4	53.1	60.8	74.3	50.9	54.7	**64.0**	63.0	52.6
Frozen (no adaptation)	38.9	37.9	40.3	34.4	28.4	41.2	42.0	48.0	55.9	62.8	73.8	29.9	49.2	52.2	56.7	46.1
Reset at every shift (episodic)	45.4	46.8	**47.0**	41.2	38.0	50.3	51.1	**54.7**	56.0	62.9	**74.8**	35.0	**57.3**	61.4	61.3	52.2

Table 22: Final test, per corruption (severity 5; accuracy of Vanilla DD, $\beta = 0.5$, %): each corruption's accuracy averaged over the orderings of the stream (the episodic reset and the frozen duo: scripts/03b rows). Bold: the best method on that corruption.

11 Limits and next steps

- **Little independent data.** The calibration pool of the screen has 4 independent rotations (the two seeds of a rotation are nearly identical) plus 2 healthy streams; at budget 1 it allows about 32 false alarms, so differences of about 20% in delay between rules are not meaningful. The four held-out rotations are the same images in another order; the five test orderings reuse the same 15 shifts and images, and the models are deterministic, so their spread understates the uncertainty. The paired standard errors treat visits as independent although a collapse carries over to the next visit: read them as a noise floor, not a confidence interval. The bootstrap intervals resample the 15 corruptions, which differ a lot in difficulty, and are wide for that reason.
- **Held-out is in-sample.** Every rate, threshold and schedule choice was made on it; the test stream was touched once, after the configurations were frozen, and nothing was changed afterwards. The claim that the test comparison is honest rests on that order (the frozen configurations are in `out/results/reset_tune/final_configs.json`).
- **One selection-code correction during the runs** (the extension cap also bounds the refinement of rule 1, Section 8); no earlier choice changed. The cap leaves `own_cusum`'s threshold at the end of its ladder: a plateau reported as such, not a located optimum.
- **Frequency against detection is answered only for one rule, one draw per test ordering.** The matched-count control was run for `own_fast_vs_slow` (five draws on held-out validation, one draw per ordering on test); the other rules and the ResNet-50's share of the effect were not controlled separately. The held-out margin over random (about +3.7) is larger than the test margin (Section 10).
- **The ResNet-50 is not helped by any `own` rule beyond what a random reset does** (Section 9); one rule with the same settings was used for both members. A per-member offline screen found no setting or family that detects the ResNet-50's collapses much earlier (previous paragraph): its nuclear-norm proxy lags or misleads, so the next candidate is another signal for the ResNet-50, not another detector. The detector's other parameters (the two averaging speeds, ν, the warm-up) stay as the screen selected them, and the schedules' periods (500 and 1,000) were fixed, not tuned.
- **The `gap` mechanism** (a reset member's proxy dip moving its partner's gap) is a reading of the figures, not a measurement; the staggered-schedule control was not run because `gap_fast_vs_slow` is not the best rule.
- **Short dips.** They account for most misses of the screen. Treating self-recovering dips under ≈ 32 batches as non-events lowers every miss rate (for the leaders, from about 0.3 to 0.2) and could change the offline ranking.
- **Not run.** Another duo, Joint DD with the best rule, and the oracle with other definitions (stage 6 of the plan; the author has not asked for it). The compute used (about 115 held-out and 40 test runs of roughly 68 minutes) is above the 150 GPU-hour estimate of the plan, because of the threshold extensions, the conservative baseline on four orderings and the five control draws.

Reproduce.

- `scripts/04c_gate_stress.py` writes the S, H and T runs (one call each, with `--extra S:1.4e-4:5e-4`, `--extra H:1.26e-5:2.2e-5`, `--extra T:1.4e-4:5e-4 --stream test`).
- `scripts/37b_screen_resets_stress.py` (`--duo vitb16_rn50v1 --bn`) runs the screen (CPU, about one minute) and writes `out/results/screen_resets_stress/`.

- `scripts/38_run_reset_live.py` runs the live conditions (GPU, about 70 minutes per run; `--conditions`, `--stream heldout/test`), `scripts/39_tab_reset.py` writes the tables and figures of Section 7.
- Code: `src/reset/screening.py`, `detectors.py`, `controller.py`.

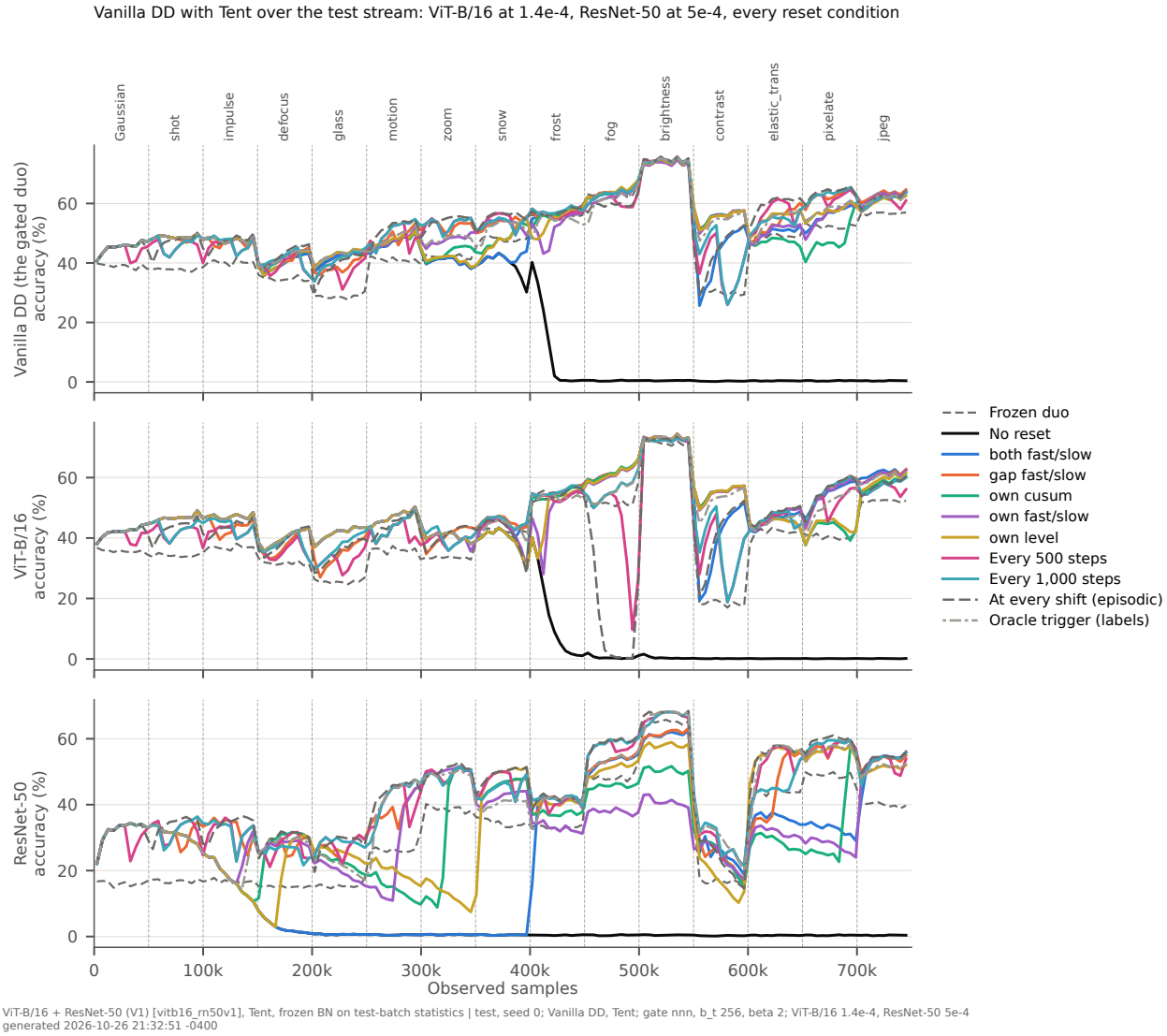


Figure 1: Test stream, seed 0: Vanilla DD (top), the ViT-B/16 and the ResNet-50 (below) under every condition, accuracy over windows of 5,120 images; the dashed grey line is the frozen duo / member, dashed lines the references (episodic, oracle). Without a reset (black) both members die (accuracy below 5% for four windows of 5,120 images): the ResNet-50 from 158,720 images, the ViT-B/16 from 430,080. `both_fast_vs_slow` (blue) leaves the ResNet-50 at 0% until about 400k.

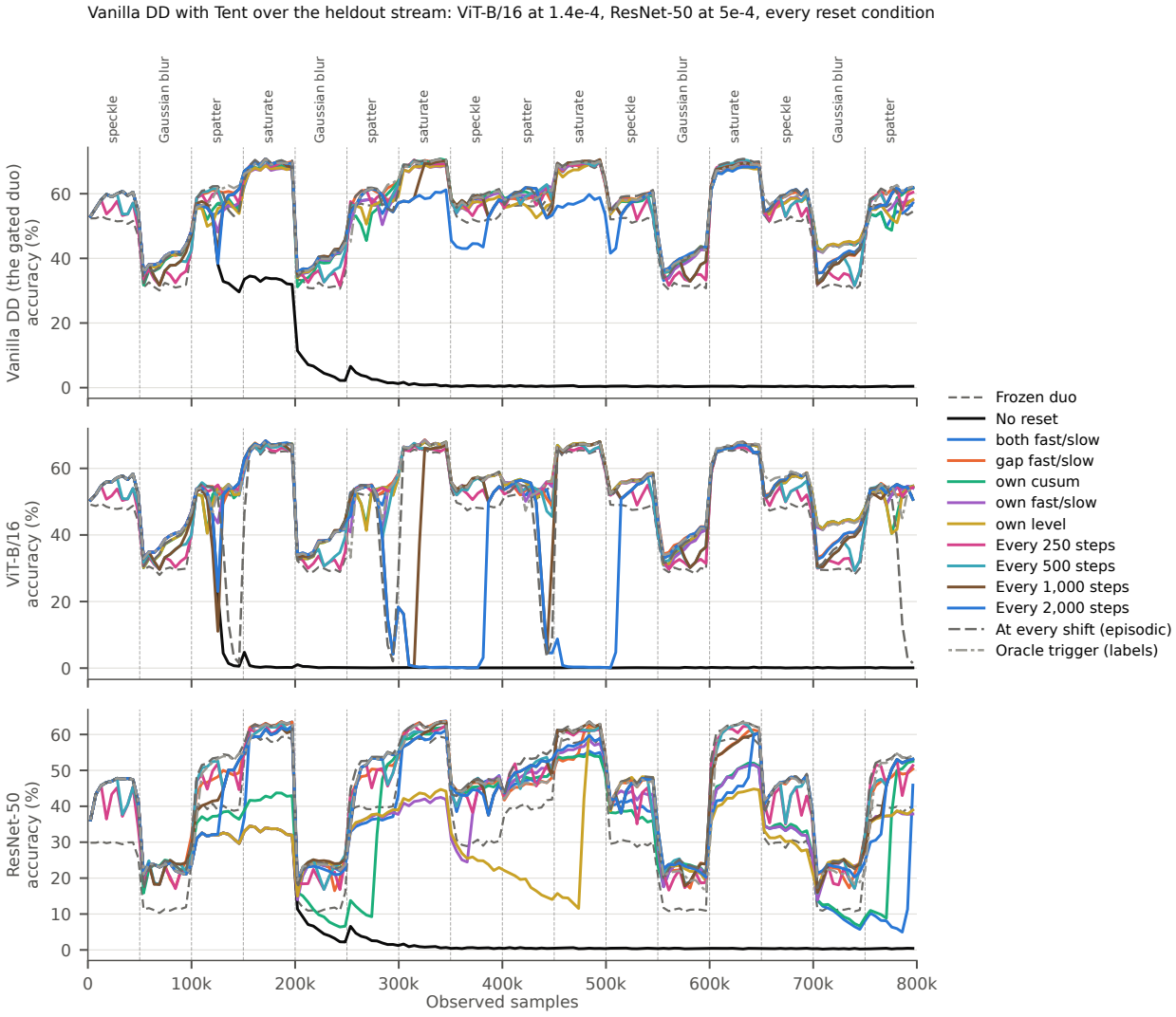


Figure 2: As Figure 1 on the held-out stream (4 shifts × 4 rounds, rotation 0).

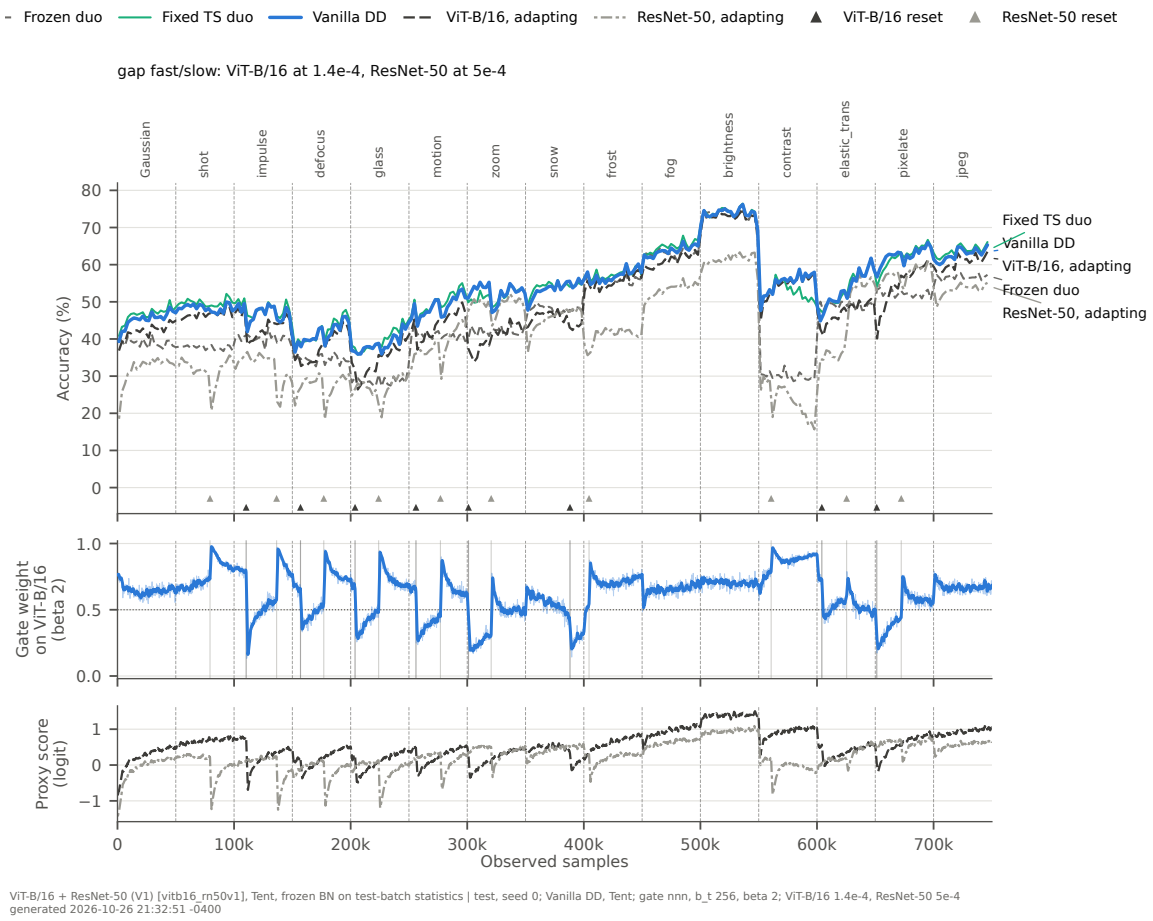


Figure 3: `gap_fast_vs_slow`, test stream, seed 0: accuracy of the frozen duo, the fixed-TS duo, Vanilla DD and both members; the gate's weight on the ViT-B/16; the proxy scores it reads. A triangle marks each reset (dark: ViT-B/16, light: ResNet-50), a line the same reset in the weight panel. The ResNet-50 is reset first and the ViT-B/16 20–69k images later, six times in a row in the first 400k images.

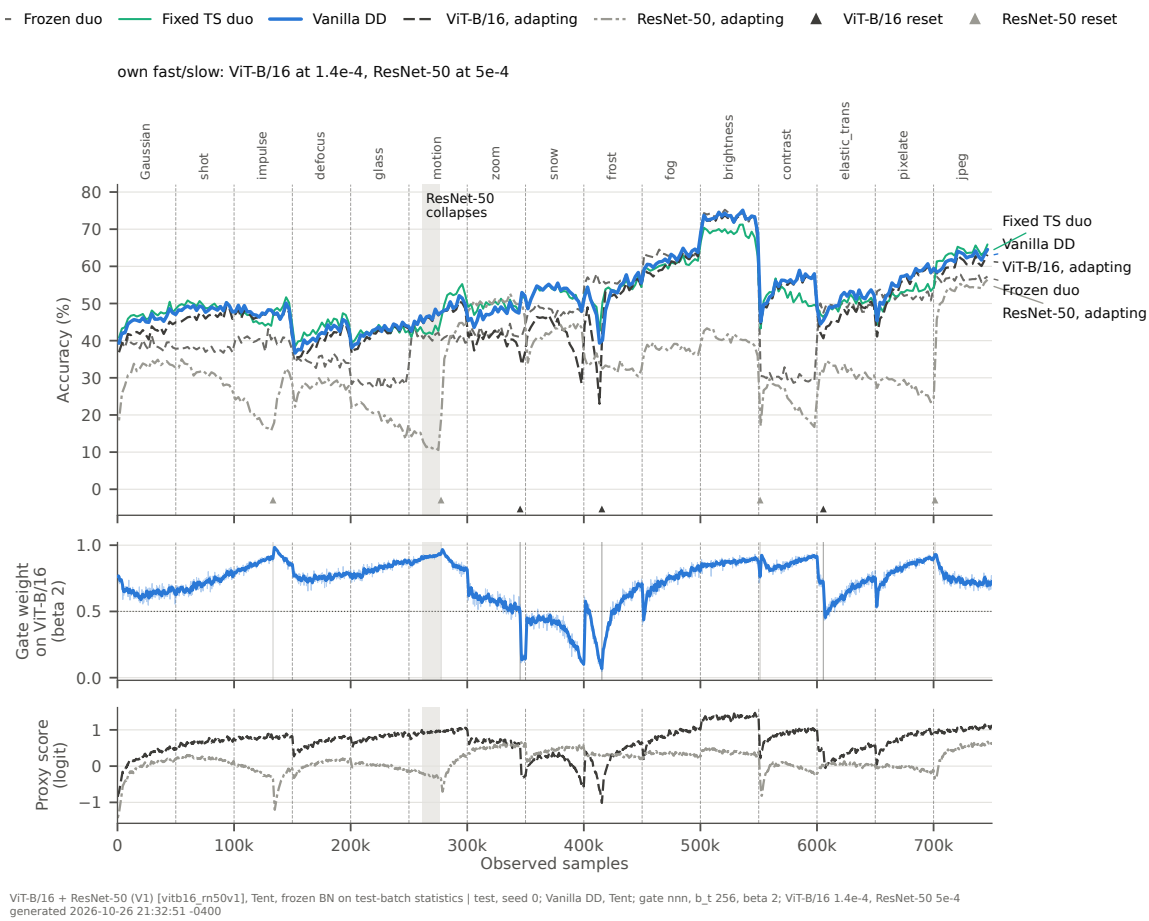


Figure 4: `own_fast_vs_slow`, test stream, seed 0, as Figure 3. It resets far less often: seven resets in all, the ResNet-50 at 133k, 278k, 551k and 701k images and the ViT-B/16 at 345k, 415k and 605k; the shaded band is the ResNet-50's collapse episode, reset at 278k. Between 278k and 551k the ResNet-50 is not reset.